CORSO DI INGEGNERIA DEL SOFTWARE — A.A. 2026/27 — DOCUMENTO 1

VoltPark — Software Project Management Plan

Sistema di Smart Parking & EV Charging con Bilanciamento Dinamico del Carico (Smart Grid Load Balancing)

Autori: Matteo Sala, Niccolò Angotti, Davide Donini **Docenti:** Prof. Angelo Gargantini, Dott.ssa Silvia Bonfanti
Versione: 1.0.0

Versione	Data	Autori	Descrizione delle modifiche
`1.0.0`	12/11/2026	M. Sala, N. Angotti, D. Donini	Stesura iniziale del Project Plan per sottomissione e approvazione tramite issue su GitHub.

1. Introduction

Il presente documento costituisce il **Project Plan** (*Documento 1 — Software Engineering Management*) del progetto **VoltPark**, sviluppato nell'ambito del corso di Ingegneria del Software (A.A. 2026/27). L'obiettivo del progetto è progettare, modellare, implementare e validare un sistema software per la gestione automatizzata di un parcheggio intelligente dotato di stalli standard e postazioni di ricarica per veicoli elettrici (EV), governato da un motore centrale di *Smart Grid Load Balancing*.

Il sistema risponde al problema reale del sovraccarico elettrico nelle infrastrutture di sosta: quando molteplici veicoli elettrici richiedono energia simultaneamente, il gestore del carico ripartisce dinamicamente la potenza disponibile tra le colonnine attive in funzione dei profili di priorità scelti dagli utenti (*Eco/Standard* oppure *Fast/Priority*), garantendo il rispetto costante della soglia massima di potenza erogabile dall'impianto. Al termine della sessione, il software calcola la tariffa complessiva integrando il costo orario di sosta, l'energia effettivamente prelevata in kWh e le eventuali penali di occupazione prolungata a carica ultimata (*overstay fee*).

> **Delimitazione del perimetro funzionale (scope management):** Per garantire che il software finale implementi la quasi totalità di quanto specificato nella documentazione entro il monte ore preventivato, il perimetro della versione `1.0.0` si concentra sui quattro pilastri del dominio: (1) gestione accessi e prenotazione stalli, (2) ciclo di vita delle sessioni di ricarica EV, (3) algoritmi di bilanciamento dinamico del carico elettrico e (4) tariffazione con calcolo delle penali di *overstay*. La simulazione di fonti energetiche ausiliarie (come i pannelli fotovoltaici) è esclusa dal perimetro iniziale e classificata come estensione futura opzionale.

2. Process Model

In conformità con gli obiettivi formativi del corso, che richiedono di dedicare almeno il **70% del tempo complessivo** alle attività di ingegneria dei requisiti, modellazione UML e progettazione architetturale, il gruppo adotta un **processo iterativo-incrementale basato su timeboxing con approccio MDA (Model-Driven Architecture)**.

Il ciclo di vita è articolato in **4 timebox** della durata fissa di due settimane ciascuno. La tecnica del *timeboxing* (mutuata dalle metodologie agili e RAD) fissa in modo rigido la data di rilascio di ciascuna

iterazione: qualora emergessero ritardi, il team ridimensiona le funzionalità accessorie preservando l'integrità architetturale e le scadenze. L'approccio combina due modalità operative complementari:

- **Approccio MDA (Model-Driven Architecture) per il dominio:** La logica centrale e la macchina a stati del sistema vengono modellate formalmente in **StarUML v5**. A partire dal *Class Diagram* di progettazione viene generato automaticamente lo scheletro iniziale delle classi Java del modulo di backend (`VP_core_db`), che viene poi raffinato manualmente mantenendo la tracciabilità bidirezionale tra modello e codice.
- **Prototipazione incrementale per l'interfaccia web:** Lo strato di presentazione (`VP_web`) viene realizzato tramite il framework **Vaadin 24** come una singola dashboard operativa (*VoltPark Control Room*), consentendo di validare rapidamente gli scenari d'uso e l'interazione con il simulatore di rete elettrica in vista della dimostrazione d'esame.

3. Organization of the Project

L'organizzazione interna del gruppo di lavoro segue il modello **decentralizzato democratico (egoless programming)** descritto nella letteratura di riferimento del corso. I tre membri del team operano su un piano paritario: le decisioni architetturali vengono prese per consenso, il codice appartiene all'intero gruppo e ogni artefatto (documentale o sorgente) viene sottoposto a revisione incrociata prima dell'integrazione. Per assicurare tuttavia ordine operativo ed equità di contributo, a ciascun componente sono assegnate responsabilità primarie ben distinte (dettagliate nella Sezione 7).

Il coordinamento con i docenti del corso avviene attraverso il repository ufficiale **GitHub**, condiviso preventivamente con il Prof. Angelo Gargantini (account `garganti`) e la Dott.ssa Silvia Bonfanti (account `silviabonfanti`). Le comunicazioni formali, inclusa la richiesta di approvazione del presente Project Plan, vengono gestite mediante l'apertura e l'assegnazione di specifiche *issue* sul repository.

La struttura fisica del repository separa rigorosamente la documentazione dal codice sorgente ed evita la presenza di file `.project` nella directory radice, suddividendo il software in due progetti Eclipse gestiti tramite Maven:

Percorso nel repository	Tipo artefatto	Contenuto e responsabilità architetturale
`VoltPark/`	Root del repository	Contiene esclusivamente il file `.gitignore`, il `README.md` e le due sottocartelle principali (nessun file `.project` nella root).
`VoltPark/documenti/`	Documentazione e modelli	Contiene i 6 documenti della relazione finale, i sorgenti documentali e il file di progetto StarUML v5 (`VoltPark_Model.mdj`).
`VoltPark/codice/VP_core_db/`	Progetto Eclipse Maven	Modulo di backend: logica di dominio, *Load Balancer*, Design Pattern, persistenza su DB embedded H2, logging Log4j2 e test JUnit 5 in `src/test/java`.
`VoltPark/codice/VP_web/`	Progetto Eclipse Maven	Modulo di frontend web: interfaccia utente *VoltPark Control Room* sviluppata in Java con Vaadin 24 e Spring Boot, dipendente unidirezionalmente da `VP_core_db`.

4. Standards, Guidelines, Procedures

Per garantire uniformità stilistica, manutenibilità e piena conformità alle regole di valutazione del corso, il team adotta i seguenti standard tecnici e procedurali:

- **Specifica dei requisiti (standard IEEE 830):** Il Documento 3 viene redatto seguendo la struttura dello standard IEEE 830, identificando univocamente ogni requisito funzionale (`RF-XX`) e non funzionale (`RNF-XX`) secondo criteri di verificabilità, non ambiguità e tracciabilità verso i casi d'uso e i test.
- **Modellazione UML 2.x:** Tutti i diagrammi obbligatori (*Use Case, Class, State Machine, Sequence, Timing, Activity, Component, Package*) vengono realizzati con **StarUML v5** e salvati in un unico modello coerente.
- **Standard di codifica Java 21 e architettura dei package:** Il codice rispetta le convenzioni di denominazione standard Java, applica rigorosamente l'*information hiding* (visibilità `private` e `protected` per gli attributi, esposizione tramite interfacce) e garantisce l'**assenza totale di cicli di dipendenza tra i package**. Nessun file sorgente o di test può contenere percorsi assoluti legati al file system locale.
- **Procedure di configuration management su GitHub:** È vietato il caricamento massivo di file a fine progetto o il commit diretto sul branch principale per modifiche sostanziali. Ogni attività nasce da una **issue** dedicata, viene sviluppata su un *feature branch* (es. `feature/load-balancer`, `docs/ieee830-requirements`) e viene integrata nel branch `main` solo tramite **pull request** sottoposta a **code review** da parte di almeno un altro membro del gruppo.
- **Configurazione `.gitignore` e progetti Eclipse:** Il file `.gitignore` esclude le directory di build generate da Maven (`target/`, `*.class`, `*.jar`), ma preserva obbligatoriamente sotto controllo di versione i file `.project`, `.classpath` e `pom.xml` di entrambi i progetti Eclipse per consentire l'importazione immediata da parte dei docenti.

5. Management Activities

Le attività di gestione del progetto accompagnano l'intero ciclo di sviluppo e comprendono:

1. **Pianificazione e controllo dei timebox:** Al termine di ciascuno dei 4 timebox bisettimanali, il team tiene una breve riunione di revisione per verificare il completamento delle issue assegnate, controllare il rispetto della ripartizione oraria 70/30 e pianificare il timebox successivo.
2. **Tracciamento tramite GitHub issue e milestone:** Tutte le attività di documentazione, modellazione UML, codifica e testing sono censite come issue su GitHub, etichettate per tipologia (`documentation`, `uml`, `enhancement`, `testing`, `bug`) e assegnate ai singoli responsabili.
3. **Monitoraggio dell'equità dei contributi individuali:** In vista della valutazione individuale prevista dal regolamento, il team monitora periodicamente le statistiche del repository (distribuzione dei commit, pull request aperte e revisionate, issue chiuse) affinché l'impegno dei tre componenti risulti bilanciato e trasparente.

6. Risks Potential

La tabella seguente riporta l'analisi preventiva dei principali rischi tecnici, progettuali e organizzativi individuati dal gruppo, classificati per probabilità di occorrenza e impatto, insieme alle relative azioni di mitigazione:

ID	Descrizione del rischio	Probabilità	Impatto	Strategia di prevenzione e mitigazione
`R-01`	Sovrapposizione del carico di lavoro con le prove in itinere e gli altri esami della sessione invernale.	Media	Alto	Adozione del timeboxing rigoroso a partire da novembre, taglio preventivo del modulo fotovoltaico e piano di contingenza già predisposto per lo slittamento dall'appello di gennaio a quello di febbraio.
`R-02`	Disallineamento tra i diagrammi UML in StarUML v5 e l'implementazione effettiva in Java.	Alta	Medio	Generazione iniziale dello scheletro del codice dal *Class Diagram* (MDA) e verifica sistematica di allineamento tra modello `.mdj` e sorgenti durante le code review e nel Timebox 4.
`R-03`	Curva di apprendimento del framework Vaadin 24 per lo sviluppo dell'interfaccia web.	Media	Medio	Progettazione di una singola vista centralizzata (*Control Room*) priva di pagine ridondanti, sviluppata in coppia da N. Angotti e D. Donini sopra API Java disaccoppiate.
`R-04`	Introduzione accidentale di cicli di dipendenza tra package o riferimenti assoluti a file locali.	Bassa	Alto	Separazione fisica in due progetti Maven (`VP_core_db` e `VP_web`), verifica statica con JDepend/Stan4J e test di esecuzione su tutti e 3 i PC del gruppo prima della consegna.
`R-05`	Conflitti di merge sul repository GitHub durante la scrittura simultanea di codice e documentazione.	Bassa	Basso	Lavoro su branch granulari associati a singole issue e modularizzazione dei sorgenti della documentazione in file separati per ciascuno dei 6 capitoli.

7. Staffing

Il team è composto da **3 membri** che operano senza ruoli fittizi, riflettendo l'organizzazione reale del lavoro svolto. Pur collaborando a tutte le fasi del progetto, ciascun componente assume la responsabilità primaria di specifici moduli architetturali, diagrammi UML e capitoli della relazione:

Componente	Responsabilità primarie (codice e architettura)	Responsabilità primarie (UML e documentazione)
Matteo Sala	Coordinamento generale del progetto, configurazione repository GitHub e build Maven, progettazione architetturale, sviluppo logica centrale in `VP_core_db` (motore di *Load Balancing* e tariffazione) e co-sviluppo test di unità.	Coordinamento e struttura dei documenti della relazione, stesura Doc 1 (*Project Plan*) e Doc 2 (*Gestione Progetto*), realizzazione di *Sequence Diagram*, *Component Diagram* e *Package Diagram*.
Niccolò Angotti	Modellazione delle entità di dominio (`Stallo`, `Colonnina`, `SessioneRicarica`), gestione degli stati, strato di persistenza su database embedded H2 in `VP_core_db` e co-sviluppo dell'interfaccia web Vaadin in `VP_web`.	Stesura Doc 3 (*Requisiti IEEE 830*) e Doc 4 (*Design*), generazione codice MDA da StarUML, realizzazione di *Use Case Diagram*, *Class Diagram* e *State Machine Diagram*.
Davide Donini	Co-sviluppo dell'interfaccia web Vaadin (*VoltPark Control Room*) in `VP_web`, progettazione ed esecuzione della suite di test JUnit 5, analisi statica delle metriche e attività di refactoring.	Stesura Doc 5 (*Testing*) e Doc 6 (*Maintenance*), calcolo complessità di McCabe e coverage, realizzazione di *Activity Diagram* e *Timing Diagram*.

8. Methods and Techniques

Il progetto mette in pratica le metodologie e le tecniche di Ingegneria del Software studiate a lezione attraverso le seguenti scelte tecniche:

- **Ingegneria dei requisiti:** Elicitazione per scenari operativi, stesura della specifica secondo lo standard **IEEE 830** e modellazione dei casi d'uso con relazioni formali di `«include»` ed `«extend»`.
- **Modellazione comportamentale e strutturale (MDA):** Utilizzo completo di **StarUML v5** per descrivere la struttura statica (con forward engineering verso Java) e la dinamica reattiva del sistema, ponendo particolare cura nello *State Machine Diagram* della colonnina (dotato di stati composti, guardie ed eventi temporali per l'*overstay*) e nel *Timing Diagram* per l'evoluzione temporale della potenza erogata.
- **Design pattern (GoF):** Applicazione di almeno tre pattern progettuali nel modulo `VP_core_db`:
  1. **Strategy Pattern:** per rendere intercambiabili a runtime sia l'algoritmo di ripartizione del carico elettrico (es. *Equal Split*, *Priority Tier VIP/Fast*, *First-Come First-Served*) sia le politiche di tariffazione;
  2. **Observer Pattern:** per notificare automaticamente il *Power Grid Monitor* a ogni avvio, variazione o completamento di una sessione di ricarica;
  3. **State Pattern:** per tradurre in modo polimorfico (tramite `@Override`) le transizioni di stato della colonnina evitando costrutti condizionali complessi.
- **Costrutti di Java moderno (Java 8–21):** Impiego estensivo di *Stream API* e *Lambda Expression* per il filtraggio degli stalli e l'aggregazione dei carichi in kW, `Record` per i DTO immutabili scambiati con la vista Vaadin, e `Optional<T>` per la gestione sicura dei risultati provenienti dal database H2.
- **Testing, complessità e refactoring:** Sviluppo di test di unità automatizzati con **JUnit 5** in cartella separata (`src/test/java`), calcolo della complessità ciclomatica di **McCabe** sui metodi algoritmici del *Load Balancer* e documentazione degli interventi di refactoring effettuati per migliorare la qualità interna del codice.

9. Quality Assurance

Il piano di *Quality Assurance* persegue quattro attributi di qualità prioritari per il dominio di *VoltPark*: **Affidabilità (Reliability)** nel rispetto del limite massimo di potenza della rete, **Manutenibilità (Maintainability)** tramite basso accoppiamento, **Testabilità (Testability)** della logica di dominio isolata dalla GUI e **Usabilità (Usability)** della dashboard di simulazione.

Attività di verifica	Strumento adottato	Soglia e criterio di accettazione
Analisi statica e code smell	**SonarLint** & **PMD**	Zero bug critici o blocker; verifica del corretto *information hiding* e contenimento della complessità ciclomatica di McCabe per singolo metodo.
Analisi dipendenze package	**JDepend** / **Stan4J**	**Zero cicli di dipendenza** tra i package di `VP_core_db` e `VP_web`; verifica delle metriche di coesione e accoppiamento.
Unit testing e coverage	**JUnit 5** & **EclEmma (JaCoCo)**	100% dei test di unità verdi prima di ogni merge su `main`; copertura del codice (*Instruction/Branch Coverage*) $\geq$ **80%** sui package di logica di dominio.
Ispezione del codice	**GitHub Pull Request**	Obbligo di *code review* approvata da almeno un altro componente del gruppo per ogni pull request diretta al branch `main`.

10. Resources

Per la realizzazione del progetto non sono richiesti servizi esterni da installare manualmente né infrastrutture proprietarie; tutte le risorse software sono open-source o gratuite e gestite in modo riproducibile tramite Maven:

- **Risorse umane:** 3 studenti del corso (Matteo Sala, Niccolò Angotti, Davide Donini), ciascuno con una disponibilità pianificata di 36 ore di lavoro effettivo.
- **Risorse hardware:** 3 workstation portatili personali per lo sviluppo collaborativo, almeno una delle quali sarà predisposta e testata per l'esecuzione della demo live e della suite JUnit durante la prova orale.
- **Risorse software e librerie:** IDE **Eclipse** (obbligatorio per la compatibilità dei progetti), **JDK 21**, **Apache Maven** per la gestione delle dipendenze, **StarUML v5** per la modellazione UML e la generazione del codice, **H2 Database** in modalità embedded per la persistenza locale, **Apache Log4j2** per il tracciamento degli eventi di sistema, **Vaadin 24** con Spring Boot per la web application, **JUnit 5** per il testing e **Git / GitHub** per il versionamento.

11. Budget and Schedule

Il budget del progetto è espresso in termini di impegno orario (ore-uomo). In linea con le indicazioni dei docenti (30–40 ore per partecipante), il team fissa un budget complessivo di **108 ore-uomo** (**36 ore a testa**), ripartite rigorosamente secondo il vincolo progettuale del corso: **75 ore (∼70%)** dedicate a pianificazione, ingegneria dei requisiti, modellazione UML, architettura e documentazione, e **33 ore (∼30%)** dedicate a implementazione Java e testing JUnit.

Avendo sostenuto la prima prova in itinere, il gruppo pianifica la schedulazione principale puntando al **primo appello utile di gennaio 2027** (con piano di contingenza già calibrato per l'appello di febbraio in caso di necessità). La tabella seguente dettaglia i 4 timebox bisettimanali e le milestone obbligatorie:

Fase / timebox	Periodo	Attività principali e deliverable prodotti	Ore team	Tipologia
Timebox 1 Planning & Requisiti	12 Nov – 26 Nov	Setup repository e progetti Maven, stesura Doc 1 (*Project Plan*), apertura issue di approvazione ai docenti (**Milestone M1: $\geq$ 1 mese prima dell'esame**), stesura Doc 3 (*Requisiti IEEE 830* e *Use Case Diagram*).	27 h (9 h/testa)	100% Modellazione & Doc
Timebox 2 Architettura & MDA	27 Nov – 12 Dic	Realizzazione completa dei diagrammi UML in StarUML v5 (*Class, State Machine, Sequence, Timing, Activity, Component, Package*), definizione Design Pattern, generazione scheletro codice MDA e stesura Doc 4 (*Design*).	33 h (11 h/testa)	100% Modellazione & Doc
Timebox 3 Sviluppo Core & Web	13 Dic – 30 Dic	Implementazione completa di `VP_core_db` (logica *Load Balancer*, stati colonnina, persistenza H2, logging Log4j2) e sviluppo della dashboard web Vaadin in `VP_web`.	21 h (7 h/testa)	100% Codifica Java
Timebox 4 Testing & Chiusura	31 Dic – 12 Gen	Sviluppo suite JUnit 5, calcolo complessità di McCabe, analisi metriche (SonarLint, PMD, JDepend), refactoring, stesura Doc 2, Doc 5 e Doc 6. **Milestone M2: Consegna 5 giorni prima dell'orale**.	27 h (9 h/testa)	12 h Testing 15 h Doc
Totale progetto	**8 Settimane**	**Ripartizione finale: 75 h Modellazione/Requisiti/Doc (69,4%) — 33 h Sviluppo/Testing (30,6%)**	**108 h**	**36 h / membro**

12. Changes

Qualsiasi modifica ai requisiti, ai modelli UML, al perimetro funzionale o alla pianificazione emersa dopo l'approvazione iniziale segue una procedura strutturata di *Change Management*:

1. **Apertura della change request:** Il membro che rileva la necessità di una modifica apre una issue su GitHub con etichetta `change-request`, descrivendo la motivazione tecnica e i componenti impattati.
2. **Analisi d'impatto e approvazione del team:** Il gruppo valuta l'impatto della richiesta sui vincoli di tempo del timebox corrente; se approvata, la modifica viene applicata prima al documento dei requisiti e al modello StarUML, e successivamente propagata al codice Java e ai test JUnit.
3. **Tracciamento delle versioni:** Ogni revisione del Project Plan o degli altri documenti viene registrata nella tabella della cronologia delle versioni posta in apertura del documento, mentre gli eventuali scostamenti rispetto alla pianificazione preventiva vengono documentati consuntivamente nel *Documento 2 (Software Life Cycle)*.

13. Delivery

La consegna e la presentazione finale del progetto avvengono secondo le scadenze e le modalità stabilite dal regolamento d'esame:

- **Approvazione preventiva del Project Plan (T − 30 giorni):** Condivisione del repository GitHub con gli account `garganti` e `silviabonfanti` e apertura di una issue assegnata al docente per l'approvazione del presente documento almeno un mese prima della data d'esame.
- **Completamento del progetto (T − 5 giorni):** Consolidamento sul branch `main` di tutti gli artefatti richiesti almeno 5 giorni prima della discussione orale:
  - I due progetti Eclipse Maven funzionanti (`codice/VP_core_db` e `codice/VP_web`) completi di file `.project`, `pom.xml` e suite di test JUnit eseguibile senza configurazioni esterne;
  - Il file di progetto StarUML v5 contenente tutti i diagrammi UML (`documenti/VoltPark_Model.mdj`);
  - La relazione finale strutturata nei 6 documenti/capitoli ufficiali (*Doc 1: Project Plan, Doc 2: Gestione del Progetto, Doc 3: Requisiti, Doc 4: Design, Doc 5: Testing, Doc 6: Maintenance*).
- **Finestra di rifinitura e discussione orale (Giorno T):** Tra i 5 giorni antecedenti e la data dell'orale sono consentite esclusivamente correzioni minori e rifiniture documentali (senza creazione di nuovi progetti Java o nuovi documenti). La discussione orale, della durata massima di 30 minuti per i 3 componenti, sarà supportata dalle slide redatte sul template ufficiale del corso e includerà l'esecuzione dal vivo del software e dei test JUnit.